This file is downloaded from Norwegian Research Information Repository. https://nva.sikt.no/

Accepted manuscript

Bos, S., Bodahl, V., Moholth, O. C., & Gundersen, H. (2025, June). REBEL-6: A 32-trit balanced ternary instruction set architecture with R2R compiler pipeline for C. In *2025 IEEE 55th International Symposium on Multiple-Valued Logic (ISMVL)* (pp. 98-103). IEEE Computer Society. https://doi.org/10.1109/ISMVL64713.2025.00028

This article has been accepted for publication and undergone full peer review but has not been through the copyediting, typesetting, pagination, and proofreading process, which may lead to differences between this version and the Version of Record. The version of record is available at IEEE: https://doi.ieeecomputersociety.org/10.1109/ISMVL64713.2025.00028



Copyright © 2025, The Author(s). This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

REBEL-6: A 32-trit balanced ternary instruction set architecture with R2R compiler pipeline for C

Steven Bos
Dept. of Science and Industry systems
University of South-Eastern Norway
Kongsberg, Norway
steven.bos@usn.no

Vetle Bodahl
Dept. of Science and Industry systems
University of South-Eastern Norway
Kongsberg, Norway
246716@usn.no

Ole Christian Moholth
Dept. of Science and Industry systems
University of South-Eastern Norway
Kongsberg, Norway
137367@usn.no

Henning Gundersen
Dept. of Science and Industry systems
University of South-Eastern Norway
Kongsberg, Norway
henning.gundersen@usn.no

***Abstract*—Is a symmetrical ternary radix more efficient for computing than an asymmetrical binary one? In this work a novel balanced ternary instruction set architecture (ISA) called REBEL-6 is presented with unique features such as three-way comparison and branching, majority voting, rounding and RV32I (binary) compatibility. An open source RV32I to REBEL (R2R) compiler pipeline for C programs is provided. The 32-trit REBEL-6 ISA can be directly compared to a 32-bit RV32I ISA. Preliminary benchmarks show a 1.4% reduction in instruction executions and 33.2% reduction in dynamic power consumption due to improvements in information density and state switching.**

***Index Terms*—ISA translation, ternary computing, radix comparison, multiple-valued logic software**

I. Introduction

The potential of a ternary compute paradigm balanced around zero was already recognized in 1950 by computer pioneer Claude Shannon [1]. He demonstrated that the *"symmetrical notation offers attractive possibilities for general-purpose computing machines"* and *"simplifies many of the circuits required to take care of signs"*. Without Dennard scaling for nearly 20 years, device-centric scaling roadmaps are slowly transitioning toward efficiency-centric scaling [2, p. 1]. The international *Beyond CMOS* effort encourages to explore higher radixes and compute paradigms [2, p. 4] warranting verification of the core ideas in Shannon's work.

Radix-3 (base-3) or three-valued logic (3VL) is the first radix in the multiple-valued logic (MVL) family and is synonymously called ternary, trinary or tri-stable in literature. A large survey by Nemati et al. on ternary adders show that about 90% are unbalanced or asymmetrical [3]. This is surprising as unbalanced ternary does not exploit the many algebraic properties, circuit optimizations and elegant information encoding that Shannon mentioned. Recent work in the field of AI [4], [5] indicates that balanced ternary signals can drastically reduce energy consumption and circuit complexity for small and large language model (LLM) training and inference.

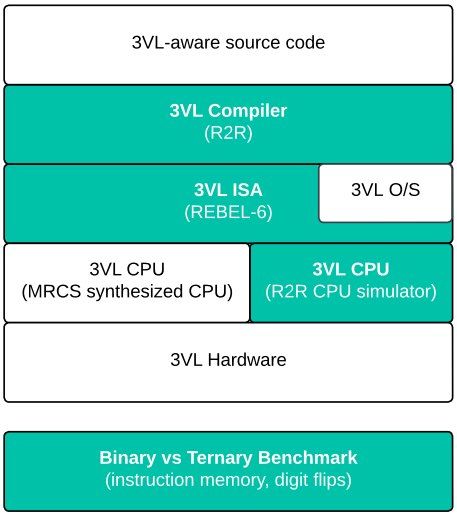


Fig. 1. Ternary (3VL) technology stack with in green the paper scope

Reduction of power consumption due to balanced ternary data paths have been reported in FPGA's [6] despite using a suboptimal binary coded ternary (BCT) implementation using 2 bits for 1 trit. In communication standards like GDDR7 [7] and USB 4.2 [8] ternary signaling based on PAM-3 have been recognized to reduce circuit complexity, improve bandwidth and power consumption. A compiled overview of the benefits of balanced ternary can be found in [9]. Note that the balanced ternary computer alphabet with logical symbols {-1,0,1} or simply {-,0,+} can still be physically represented and processed as unbalanced levels such as $\{0\ \text{V}, \frac{V_{DD}}{2}, V_{DD}\}$ or $\{1\ k\Omega, 15\ k\Omega, 100\ k\Omega\}$ with emerging devices like memristors [10] and CNTFET [11].

In this paper we address a large knowledge gap in ternary literature: an open software balanced ternary compiler pipeline and hardware-software contract for real-world applications.

Without multiple-valued hardware-aware software, ternary VLSI hardware will never utilize its theoretical potential. The scope of this paper is shown in Fig. 1 which exists of three parts. In section II REBEL-6 is introduced, the successor to REBEL-2 a minimalistic balanced ternary ISA designed for the classroom [9, p. 71]. In section III R2R is introduced, a RISC-V to REBEL compiler toolchain and REBEL CPU simulator based on the MSc thesis of Bodahl [12]. In section IV the methodology is presented to verify correct implementation and benchmark the REBEL-6 ISA and R2R software. All source code, benchmarks and tests including the modified Dhrystone version are open with permissive MIT license allowing integration in other tooling and replication of our results.

II. REBEL-6 Instruction Set Architecture

In [9, p. 71-76] REBEL-2 was introduced, a ternary first ISA and CPU design inspired by RV32I and Brousentsov's ternary principles [13]. The acronym REBEL stands for *RISC-V-like energy-efficient balanced ternary logic*. The number postfix denotes the 2-trit wide memory address of the registers. The ISA was intended for educational purposes and is designed to be minimal with just 2-trit wide register size and 9 registers. The REBEL ISA broke with the common tradition to design ternary ISA's like binary ISA's using factors of 3 instead of 2 for register and instruction size. REBEL-2 uses 10-trit instructions with four 2-trit operands, allowing 3 input and 1 output instructions.

TABLE I
SUMMARY OF RV32I, REBEL-2 AND REBEL-6

	RV32I	REBEL-2	REBEL-6
Radix	Binary	Ternary	Ternary
Instruction count	40	9[1]	63[2]
Instruction width	32 bits	10 trits	32 trits
Operand count	2-3	2-4	2-4
Register count	32	9	729
Register size	32 bits	2 trits	24 trits
PC jump per instr.	4	1	1

[1] The 9 compressed instructions cover 27/40 RV32I [9, p. 76]
[2] 29 ternary instructions plus 27 backward compatibility with RV32I instructions [12] and 7 candidate instructions in II.

REBEL-6 was made by the authors during the Master projects of Vetle Bodahl and Ole Christian Moholth and is described in more detail in [12]. The ISA defines a 6-trit wide memory address of the registers, allowing $3^6 = 729$ registers. This is a strong departure from standard sizes such as 16 in RV32E or 32 in RV32I. With more and higher radix registers better data locality can be achieved, reducing the overhead of communication with external memory. Communication and not computation has for several decades been the largest bottleneck and with newer technology nodes this problem worsens [14]. REBEL-6 builds on the philosophy and lessons learned from REBEL-2. A comparison between RV32I and the two REBEL family members is shown in Table I. Prominent features besides more registers are larger opcodes for more instructions and full RV32I compatibility while using ternary valued registers which allow 58.5% more information. This requires hardware radix conversion circuitry and a dual binary ternary ALU described in [12, p. 40] to deal with intentional binary behavior such as bit shifts and overflow when processing binary RV32I instructions.

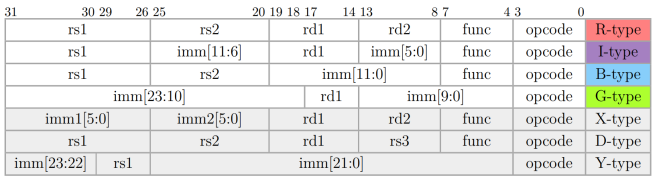


Fig. 2. REBEL-6 instruction formats

The seven instruction formats are shown in Fig. 2 and are largely inspired by the RV32I spec. The 32-trit size is not coincidentally identical to the 32-bits of RV32I. The same number of memory elements but in a higher radix allows for elegant and automated conversions. A detailed explanation and rationale for the instruction formats are described in [12]. Most notable is the distinction between instruction formats X,D and Y which are ternary only formats and the R,I,B,G instruction formats which are also used for binary compatibility. Another important choice was related to the non standardized ternary trytes and words. We choose to use 6-trit trytes and 4-tryte (24-trit) ternary words for practical reasons as 6 trits encompass a whole byte. We could define 21 trit words which fit 32 bit unsigned numbers, however a structure of 4 trytes in a word simplifies the ISA and binary to ternary translation.

TABLE II
CANDIDATE REBEL-6 INSTRUCTIONS

Instruction	Name	Type	Description
majv.t rd1, rs1, rs2, rs3	Majority vote	D	cnt(n, (rs1, rs2, rs3)) >= 2: rd = n
minv.t rd1, rs1, rs2, rs3	Minority vote	D	cnt(n, (rs1, rs2, rs3)) == 1: rd = n
cmp.t rd, rs1, rs2	Compare	R	rs1 > rs2 : rd1 = 1 rs1 == rs2 : rd1 = 0 rs1 < rs2 : rd1 = -1
li2.t rd1, rd2, imm6a, imm6b	Load two immediates	X	rd1 = imm6a rd2 = imm6b
sti.t rd1, rs1	Standard ternary inversion	R	rs1[i]==1 : rd1[i] = -1 rs1[i]==0 : rd1[i] = 0 rs1[i]==-1: rd1[i] = 1
lw.t rd1, imm24	Load word	G	rd1 = mem[imm24]
sw.t rs1, imm24	Store word	Y	mem[imm24] = rs1

The REBEL-6 spec is not yet frozen. Seven candidate instructions are shown in Table II. They are specifically designed to exploit the balanced ternary numbering system such as majority and minority voting, three-way comparison, loading two six-trit immediates in one instruction, the standard ternary inverter (STI), and loading/storing a full word. The full list of 56 REBEL-6 instructions feature balanced ternary instructions such as rounding using shifts and MIN/MAX operations and can be found in [12].

III. R2R: A RV32I TO REBEL COMPILER PIPELINE

In [12] a compiler pipeline is proposed where a single-file program in C is compiled to REBEL-6 instructions. In addition to using the RISC-V Toolchain, an open source software application called RV32I To REBEL (R2R) [15] is created. The R2R tasks are to translate binary RV32I instructions to binary and ternary REBEL-6 instructions, to output custom machine code readable by MRCS [16], and to simulate RV32I and REBEL-6 instruction execution to generate run statistics.

Taking inspiration from related works [17] [18] [19] and the general build pipeline described in [20, Ch. 2.12], we conclude that the R2R software has to include a *parser*, a *translator*, an *assembler* and a *simulator*. The R2R software design overview is illustrated in Figure 3, where these components are separated into three sections. The *frontend* handles input parsing and translation preparation. The *backend* performs the translation and the necessary operations to generate MRCS-readable machine code, and prepares the assembly code for simulation. Lastly, the *simulation* simulates instruction execution and generates run statistics. The generation of RV32I assembly, performed by a GCC compiler installed with the RISC-V Toolchain, is added to the illustration for context and the completeness of a full C to REBEL-6 pipeline. To make simulation easier and more adaptable to REBEL-6 changes, we designed the R2R software so that it operates on assembly code from frontend to simulation.

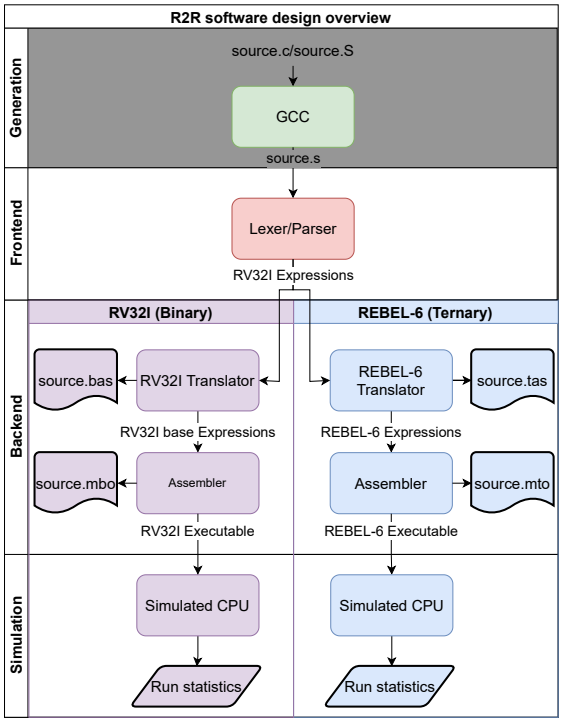


Fig. 3. The proposed RV32I to REBEL (R2R) compiler pipeline for C programs. Generated instructions allow for direct binary to ternary comparison

A. Frontend

The frontend is tasked with reading and understanding the input RV32I assembly, and outputting a structured list of identified expressions in the same order as in the input RV32I assembly file, for further use in the backend. Each expression is categorized as either an *instruction*, a *label*, or an *assembler directive*. The definition of an RV32I instruction is any RV32I base or pseudoinstruction. Labels are defined as "a symbol immediately followed by a colon ':'" [21]. Assembler directives are defined by how they begin with a "." and they're not labels (labels can also start with a "."). Listing 1 illustrates the identification of expressions.

```
.text               --> Assembler Directive
.L1:                --> Label
addi x1, x1, -1     --> Instruction
bne  x1, x0, .L1    --> Instruction
```

Listing 1. Frontend expression identification

B. Backend

The backend is responsible for generating instruction-level, code-internal executables readable by the R2R simulators from the structured expressions received from the frontend. It consists of a set of translators and a set of custom assembly-level assemblers; one of each for the RV32I path, and one of each for the REBEL-6 path. The expressions generated by the frontend are first used to generate two new sets of expressions: One where the RV32I pseudoinstructions have been resolved, and one where the RV32I instructions have been translated to REBEL-6. These expressions are then fed to their related assemblers, responsible for resolving address references and setting up a simulated memory structure. The backend output are two code-internal executables fed to their respective CPU simulators and MRCS-readable object files.

1) Translators: The translators are responsible for translating RV32I assembly code to RV32I assembly code where pseudoinstructions have been resolved according to [22, Ch. 25], and to REBEL-6 instructions as described in [15, p. 109]. Following the Translation by Transliteration and Refinement (TTR) approach described in [23], the translation is performed one line at a time. We refrain from adding an optimization step to prevent adding bias that may favor one ISA over another. The translations are then outputted to binary (.bas) and ternary (.tas) assembly files, with the intention of being able to compare them to the original RV32I assembly file. Listing 2 illustrates the translation process from RV32I to REBEL-6. Assembler directives and labels do not go through a translation process, but are kept as is.

```
Ass. Dir.:     .text            --> .text
Label:         .L1:             --> .L1
Instruction:   addi x1, x1, -1  --> addi x1, x1, -1
Instruction:   bne  x1, x0, .L1 --> bne.t x1, x0, .L1
```

Listing 2. Backend instruction translation from RV32I to REBEL-6

2) Assemblers: The custom assembly-level assemblers are responsible for resolving all address references and setting up a memory structure based on the list of translated expressions

they take as input. They output custom assembly-level, code-internal executables consisting of assembly instructions and a simulated data memory segment that the CPU simulators can operate on. Once the assemblers have resolved all labels and addresses, they also output the instructions as MRCS-readable machine code (currently only implemented for RV32I .mbo). Listing 3 illustrates how the REBEL-6 assembler resolves the address of label *.L1* and replaces its use in the *bne.t* call with the actual address, and how the assembler allocates one tryte (for assembler directives related to memory allocations, each binary memory allocation allocates the ternary equivalent) on the heap part of the simulated data memory segment before setting its value. Note how the hdata label has the address value 0x00. This is to indicate how a reference to the hdata label will point to data memory address 0x00, which due to the Harvard architecture is separate from the instruction memory address 0x00 that the .L1 label points to. Whereas one jump in REBEL-6 instruction memory is 32 trits, a jump in data memory is six trits, or one tryte.

```
Ass. Dir.:          .text              -->
Label:        0x00  .L1:               -->
Instruction:  0x00  addi x1, x1, -1    --> addi x1, x1, -1
Instruction:  0x01  bne.t x1, x0, .L1  --> bne.t x1, x0, 0x00
Ass. Dir.:          .data              -->
Label:        0x00  hdata:             -->
Ass. Dir.:          .byte 0xF1         --> On heap, allocate
                                           1 tryte = 0xF1
```

Listing 3. Backend REBEL-6 assembler resolving expressions

C. Simulation

The purpose of the assembly-level simulators is to be used for verification and benchmarking. They include both a loop used to execute instructions, and a simulated registry. They take as their input the assembly-level, code-internal executables outputted by the assemblers. The simulators then begins executing instructions, starting at PC = 0x00 or at the address of the main routine, if found, generating statistics used for benchmarking while simulating program execution.

IV. Evaluation setup

A. Verification

For verification we use the RISC-V official test code [24], consisting of 37 test files, one for each RV32I instruction, and no external dependencies. Some modifications of the test code (described in [12, p. 129-131]) is required to enable R2R to parse it, but the test values remain unchanged. The tests are used to verify that both the RV32I and REBEL-6 assembly code translation and simulation performed by R2R work as intended. The modified RISC-V tests sources used for verification, and the RV32I assembly code generated from them, is found at [15].

B. Benchmarking

[20, Ch.1.6] describes how to measure CPU performance, with a copy of the final performance measuring equation from [20, p. 36] shown in Equation 1, and further detailed in [9, Ch. 2.5.1]. We measure the number of instructions executed per program, as this measurement requires no assumptions or any relationship to hardware. In addition, to further delve into the energy cost (and from that, the heat generation) of a program, we measure the amount of digit flips performed by a program when storing data to a register, counting only state changes. We are mostly interested in the relative digit flip difference in percentages between a program compiled to a binary RV32I architecture and the same program compiled to a ternary REBEL-6 architecture. For simplicity, each digit flip is counted equally although in ternary 4 out of 6 state switches are actually half swings and typically requires less energy. RAM and databus digit flips are ignored.

$$Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} * \frac{ClockCycles}{Instruction} * \frac{Seconds}{Clockcycle} \quad (1)$$

For benchmarking we use Dhrystone [25] 2.1, a C-coded program used for measuring the performance of CPUs. As Dhrystone has some external dependencies, the code has to be modified as described in [12, ch. 6.3]. The modified source and generated RV32I assembly code can be found at [15]. In addition to Dhrystone, we use the verification code from subsection IV-A for further benchmarking. Generating vastly smaller assembly files aimed mainly towards edge cases for each individual RV32I instruction, the benchmarking results they generate must be taken with a grain of salt.

V. Results

A. Verification

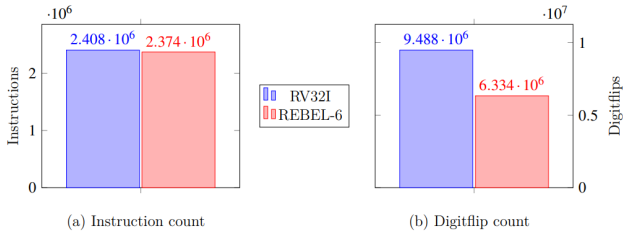
TABLE III
VERIFICATION RESULTS

Test	RV32I	REBEL-6	Test	RV32I	REBEL-6
add	✓	✓	lui	✓	✓
sub	✓	✓	auipc	✓	✓
sll	✓	✓	beq	✓	✓
srl	✓	✓	bne	✓	✓
sra	✓	✓	blt	✓	✓
slt	✓	✓	bltu	✓	✓
sltu	✓	✓	bge	✓	✓
or	✓	✓	bgeu	✓	✓
xor	✓	✓	jal	✓	✓
and	✓	✓	jalr	✓	✓
addi	✓	✓	lw	✓	✓
slli	✓	✓	lh	✓	✓
srli	✓	✓	lb	✓	✓
srai	✓	✓	lhu	✓	✓
slti	✓	✓	lbu	✓	✓
sltiu	✓	✓	sw	✓	✓
ori	✓	✓	sh	✓	✓
xori	✓	✓	sb	✓	✓
andi	✓	✓			

The verification results are shown in Table III. Each instruction corresponds to a test file related to the instruction. As can be seen, each test passed both for RV32I and for REBEL-6. With every test passing we can conclude that the R2R software is able to correctly translate and simulate all RV32I instructions and their REBEL-6 translation equivalents.

B. Benchmarks

1) Dhrystone: The run statistics generated by using the Dhrystone assembly code as input for R2R is presented in Figure 4. For the REBEL-6 executed instruction count a minor ĩ.4% improvement can be observed compared to RV32I. For the digit flips metric REBEL-6 shows a distinct reduction of 33.24% compared to RV32I. In other words, the dynamic power consumption generated by flipping the value of transistors has the potential of being decreased by around 33.24%. More so, a higher information density also has the potential to reduce static power consumption (leakage current) by using fewer transistors but this is implementation technology dependent. Our results indicate an overall reduction of power consumption using REBEL-6 (ternary) over RV32I (binary).



(a) Instruction count (b) Digitflip count

Fig. 4. Results from running the Dhrystone C-code through the proposed pipeline.

2) RISC-V Tests: With each RISC-V test being useful for verification, we decided to also use them for less formal benchmarking. Each test targets a specific instruction but includes other instructions. These tests are short and may use unrealistic test values for real world scenarios. Most writes to the registers performed in each subtest overwrites values from previous subtests rather than using pristine values from within the subtests. Since only digit flips are counted, we can not justify drawing a definitive conclusion from these results alone. Nevertheless, there are indications of an advantage using ternary over binary, particularly in cases where positive and negative numbers interchange frequently. We add the results here as a gateway for further investigation. Table IV summarizes the relative differences in percentages from the executed instruction count and digit flips metrics.

TABLE IV
THE DIFFERENCE IN MEASURED RESOURCE USE OF REBEL-6 COMPARED TO RV32I WHEN RUNNING THE RISC-V TEST FILES THROUGH THE PROPOSED PIPELINE.

Test	Run instr.	Digit flips	Test	Run instr.	Digit flips
add	-5.03%	-48.62%	lui	0.00%	-51.98%
sub	-4.66%	-53.44%	auipc	-23.08%	-42.95%
sll	-7.19%	+25.17%	beq	0.00%	-64.89%
srl	-14.42%	+28.96%	bne	0.00%	-67.71%
sra	-15.40%	-29.83%	blt	0.00%	-64.49%
slt	-2.81%	-49.42%	bltu	-16.50%	-89.71%
sltu	-2.81%	-49.65%	bge	0.00%	-74.38%
or	-22.44%	-58.11%	bgeu	-15.58%	-90.13%
xor	-22.27%	-54.51%	jal	-5.88%	-11.11%
and	-22.10%	-51.79%	jalr	-14.29%	-52.38%
addi	-6.70%	-51.15%	lw	-22.04%	-53.37%
slli	-8.45%	+19.13%	lh	-17.32%	-76.02%
srli	-16.67%	+17.56%	lb	-11.16%	-86.16%
srai	-18.07%	-38.10%	lhu	-20.42%	-67.33%
slti	-3.47%	-39.95%	lbu	-11.16%	-62.88%
sltiu	-3.47%	-39.75%	sw	-23.32%	-41.63%
ori	-16.17%	-49.31%	sh	-21.54%	-62.19%
xori	-17.16%	-40.94%	sb	-11.54%	-71.68%
andi	-12.50%	-41.00%			

VI. Discussion

How important is binary compatibility for a ternary ISA? Choosing a discrete binary optimal architecture (factor of 2) will always lead to a disadvantage as discussed in [9, p. 32-33]. We acknowledge that transitioning code from binary to ternary is not a trivial matter as some code expects binary arithmetic and overflow behavior [12, p. 85]. The amount of RV32I programs available are vast and full compatibility makes the REBEL-6 ISA with dual binary-ternary ALU implementation a running engine with some benefits shown in the results. The ISA offers an upgrade path with pure ternary instructions for code that does not rely on binary behavior. Mixing binary and ternary instructions has demonstrated benefits in other work [6], [18]. Our result also align with the benefits reported in [17] although we use a different (and open) modified Dhrystone version and much more comprehensive balanced ternary ISA. Compared to the 9 trit ISA's in [26] (with 4 instructions) and [17] (with 24 instructions) we designed a 32 trit ISA with 63 instructions. No other investigated work presents a fully open source C-to-ternary pipeline. Comparing our results with similar work is difficult with how we use a modified version of Dhrystone generating our own simulation statistics, and how other papers like [17], [18] use closed source or incomparable benchmarking code and methods. In general, more real-world, system-wide benchmarks are needed beyond Dhrystone as discussed in [27]. Common algorithms for searching and sorting which rely on three-way comparison should see power benefits from single instruction hardware execution. An elegant programming symbol for hardware accelerated comparison could be the spaceship operator as found in c++. Larger adoption of 3VL ISA's would require 3VL-aware source code and a 3VL compiler. The balanced ternary paradigm with variables trit and tryte must be standardized and become first-class citizens in modern programming languages.

VII. Conclusion

This paper contains several important contributions towards a complete ternary technology stack. This stack is essential for practical adoption of the ternary compute paradigm as the software side is often neglected or simplified in ternary literature. REBEL-6, the successor to REBEL-2 is introduced which to the best of our knowledge is the most comprehensive balanced ternary ISA targeting real-world applications. The ISA is a novel 32-trit balanced ternary ISA for efficient ternary operations and features 32-bit RV32I (binary) compatibility for direct binary versus ternary benchmarking. In addition R2R is introduced, an open source RV32I to REBEL compiler

pipeline for C programs using the RISC-V toolchain as a single dependency. The software suite can translate C programs to ternary assembly and machine code and exploits REBEL-6 instruction formats to reduce instruction count. The R2R REBEL-6 CPU simulator passes all RV32I tests demonstrating full binary compatibility. In a preliminary direct comparison using a modified Dhrystone version with the R2R software, $\tilde{1}.4\%$ fewer instructions were executed and $\tilde{3}3.2\%$ less dynamic power consumption was measured as state changes. This first result provides evidence that the symmetrical ternary radix can be more efficient for computing than an asymmetrical binary one.

VIII. Future work

Linker functionality is needed to support external dependencies such as the Newlib library and enable reusable ternary compilation units. Instruction count can most likely be further reduced as REBEL-6 supports full address space load and stores instruction formats. A ternary standard library for C would aid the compiler in communicating programmer's intent and have software constructs for unique ternary hardware capabilities. Lastly, integration of the compiler pipeline in a ternary EDA tool like MRCS would enable verification at the hardware level with realistic workloads. Only then can Shannon's radix supremacy claims be verified at large.

Acknowledgment

This research was fully funded by the University of South-Eastern Norway (USN). We thank the members of USN Ternary Research Group for their feedback on this paper.

References

[1] C. E. Shannon, "A symmetrical notation for numbers," *The American Mathematical Monthly*, vol. 57, no. 2, pp. 90–93, 1950, https://doi.org/10.1080/00029890.1950.11999490.

[2] B. C. team, "International Roadmap for Devices and Systems: 2022 update Beyond CMOS and emerging materials integration," 2022. [Online]. Available: https://irds.ieee.org/images/files/pdf/2022/2022IRDS_BC.pdf

[3] S. Nemati, M. Haghi Kashani, and R. Faghih Mirzaee, "Comprehensive survey of Ternary Full Adders: Statistics, corrections, and assessments," *IET Circuits, Devices & Systems*, vol. 17, no. 3, pp. 111–134, 2023, https://doi.org/10.1049/cds2.12152.

[4] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The era of 1-bit llms: All large language models are in 1.58 bits," 2024. [Online]. Available: https://arxiv.org/abs/2402.17764

[5] J. Nielsen and P. Schneider-Kamp, "Bitnet b1.58 reloaded: State-of-the-art performance also on smaller networks," 2024. [Online]. Available: https://arxiv.org/abs/2407.09527

[6] M. Reichenbach, J. Knödtel, S. Rachuj, and D. Fey, "RISC-V3: A RISC-V compatible CPU with a data path based on redundant number systems," *IEEE Access*, vol. 9, pp. 43 684–43 700, 2021, https://doi.org/10.1109/ACCESS.2021.3063238.

[7] J. Yang, H. Ko, K. Kim, H. Park, J. Park, J.-H. Kang, J. Cha, S. Kim, Y. Kim, M. Park, G. Lee, K. Lee, S. Lee, G. Jeon, S. Jeong, Y. Joo, J. Cha, S. Hwang, B. Kim, S. Byeon, S. Lee, H. Park, J. Cho, and J. Kim, "13.1 a 35.4gb/s/pin 16gb gddr7 with a low-power clocking architecture and pam3 io circuitry," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 232–234, https://doi.org/10.1109/ISSCC49657.2024.10454560.

[8] USB-IF Electrical Workgroup, "USB 80G PHY layer analysis," 2023. [Online]. Available: https://www.usb.org/document-library/usb-80g-phy-layer-analysis

[9] S. Bos, "Beyond 0 and 1: A mixed radix design and verification workflow for modern ternary computers," Ph.D. dissertation, University of South-Eastern Norway (USN), 2024. [Online]. Available: https://hdl.handle.net/11250/3127984

[10] J. Yang, H. Lee, J. H. Jeong, T. Kim, S.-H. Lee, and T. Song, "Circuit-level exploration of ternary logic using memristors and mosfets," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 2, pp. 707–720, 2022, https://doi.org/10.1109/TCSI.2021.3121437.

[11] S. Kim, S.-Y. Lee, S. Park, K. R. Kim, and S. Kang, "A logic synthesis methodology for low-power ternary logic circuits," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 9, pp. 3138–3151, 2020, https://doi.org/10.1109/TCSI.2020.2990748.

[12] V. Bodahl, "Rv32i to rebel (r2r): A ternary compiler pipeline for c programs," Master's thesis, University of South-Eastern Norway (USN), 2024. [Online]. Available: https://openarchive.usn.no/usn-xmlui/handle/11250/3169529

[13] A. Stakhov, "Brousentsov's Ternary Principle, Bergman's Number System and Ternary Mirror-symmetrical Arithmetic," *The Computer Journal*, vol. 45, no. 2, pp. 221–236, 01 2002, https://doi.org/10.1093/comjnl/45.2.221.

[14] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: efficient inference engine on compressed deep neural network," in *Proceedings of the 43rd International Symposium on Computer Architecture*, ser. ISCA '16. IEEE Press, 2016, p. 243–254. [Online]. Available: https://doi.org/10.1109/ISCA.2016.30

[15] V. Bodahl. (2024) RV32IToREBEL. [Online]. Available: https://github.com/Soppe/RV32IToREBEL

[16] S. Bos, H. N. Risto, and H. Gundersen, "Beyond CMOS: Ternary and mixed radix CNTFET circuit design, simulation and verification," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 80–85, https://doi.org/10.1109/ISCAS48785.2022.9937259.

[17] D. Kam, J. G. Min, J. Yoon, S. Kim, S. Kang, and Y. Lee, "Design and Evaluation Frameworks for Advanced RISC-based Ternary Processor," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022, pp. 1077–1082, https://doi.org/10.23919/DATE54114.2022.9774584.

[18] B. Cambou, P. G. Flikkema, J. Palmer, D. Telesca, and C. Philabaum, "Can Ternary Computing Improve Information Assurance?" *Cryptography*, vol. 2, no. 1, 2018, https://doi.org/10.3390/cryptography2010006.

[19] D. K. Dennis, A. Priyam, S. S. Virk, S. Agrawal, T. Sharma, A. Mondal, and K. C. Ray, "Single cycle RISC-V micro architecture processor and its FPGA prototype," in *2017 7th International Symposium on Embedded Computing and System Design (ISED)*, 2017, pp. 1–5, https://doi.org/10.1109/ISED.2017.8303926.

[20] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann Publishers Inc., 2021, isbn: 9780128203316.

[21] "Labels." [Online]. Available: https://sourceware.org/binutils/docs/as/Labels.html

[22] *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213*, RISC-V Foundation, 12 2019. [Online]. Available: https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf

[23] J. P. Cempron, C. Salinas, J. Benedict Gonzales, R. Uy, and Y. Hayakawa, "A static transliteration approach for assembly language translation," in *2016 IEEE Region 10 Symposium (TENSYMP)*, 2016, pp. 332–336, https://doi.org/10.1109/TENCONSpring.2016.7519428.

[24] "RISC-V Tests repository." [Online]. Available: https://github.com/riscv/riscv-tests

[25] R. P. Weicker, "Dhrystone: a synthetic systems programming benchmark," *Commun. ACM*, vol. 27, no. 10, p. 1013–1030, 10 1984, https://doi.org/10.1145/358274.358283.

[26] S. Gadgil, G. N. Sandesh, and chetan Kumar V, "Design and Implementation of a CNTFET-Based Ternary Logic Processor," 3 2023, preprint. [Online]. Available: https://www.techrxiv.org/articles/preprint/Design_and_Implementation_of_a_CNTFET-Based_Ternary_Logic_Processor/22259437

[27] D. Patterson, J. Bennett, P. Dabbelt, C. Garlati, G. S. Madhusudan, and T. Mudge, "Embench™: Recruiting for the long overdue and deserved demise of dhrystone as a benchmark for embedded computing," 2019. [Online]. Available: https://www.sigarch.org/embench-recruiting-for-the-long-overdue-and-deserved-demise-of-dhrystone-as-a-benchmark-for-embedded-computing/